TP1 : L'environnement Django-Python, le premier pas

Objectif du TP :

1. **Installer Python**
2. **Installer un environnement virtuel pour un projet Django**
3. **Installer Django**
4. **Créer un projet Django**
5. **Créer une base de données SQLite3 pour le projet Django**
6. **Lancer le serveur web de développement de Django**

Introduction

Le framework Django est une boite à outils open source qui permet la création des sites web en toute sécurité et performance. Il est écrit en langage **python**. C'est un framework qui fonctionne selon l'architecture MVT (Models, Vues, Templates) offrant multiples avantages et services.

Etape 1 : Télécharger et installer python

Aller à l'URL : https://www.python.org/downloads/

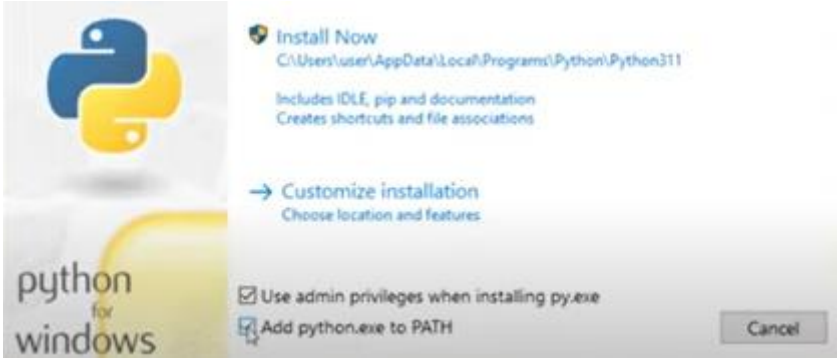
Puis, cliquer sur le bouton

1. Installation de l'environnement

a. Téléchargez et installez Python depuis python.org

- Version 3.10.2 ou ultérieure recommandée

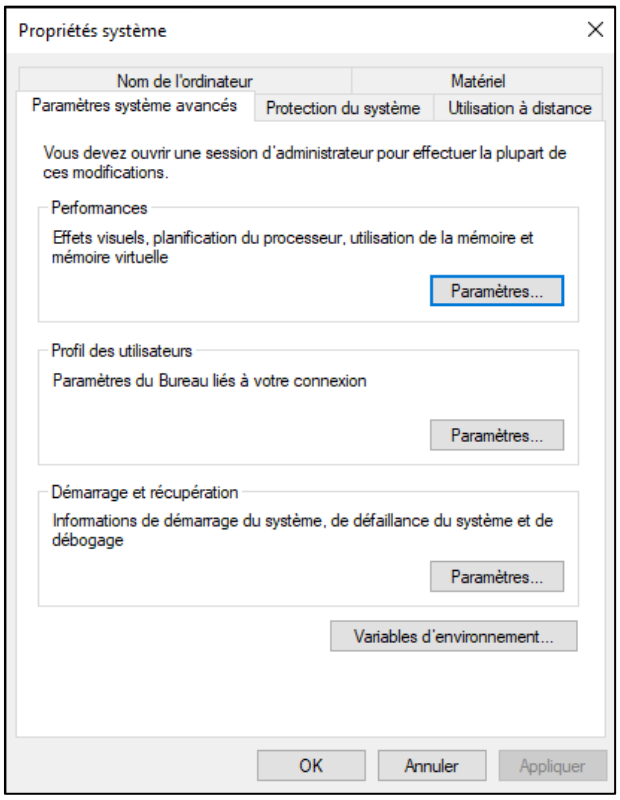
- Cochez l'option "Add Python to PATH" lors de l'installation

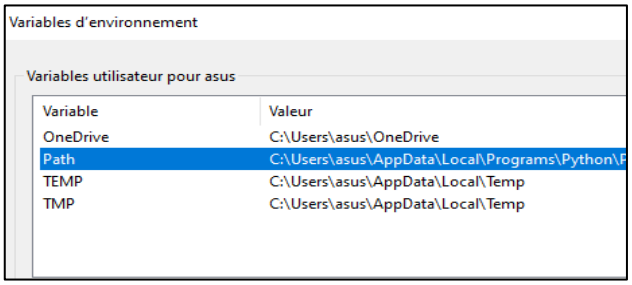


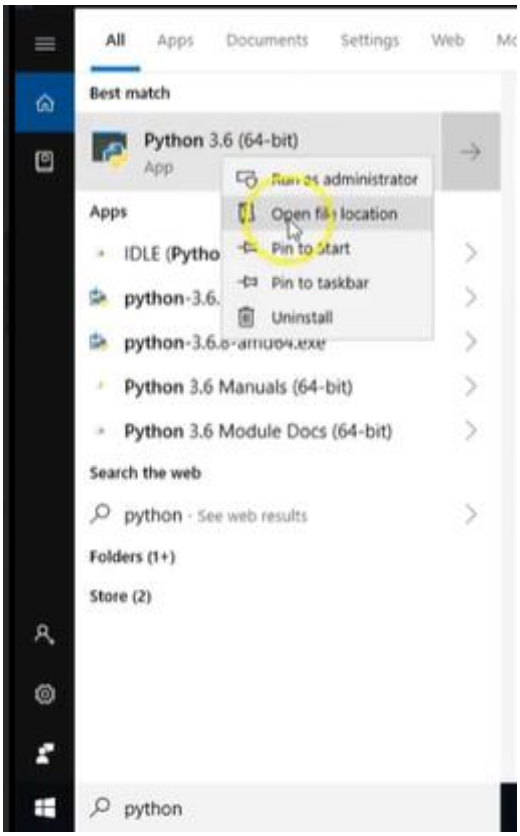
b. Ajouter Python à la variable d'environnement PATH (Windows)

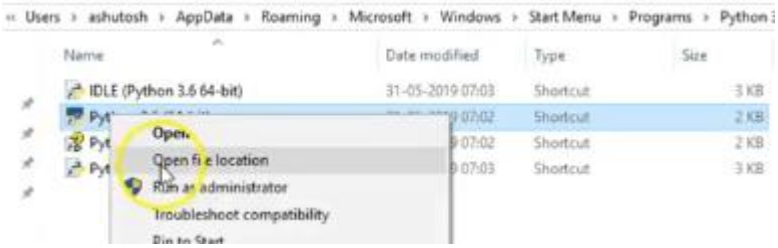
1. Clique **Démarrer** → tape variables d'environnement.
2. Clique sur **Modifier les variables d'environnement système**.
3. Dans la fenêtre → clique sur **Variables d'environnement**.
4. Dans la section **Variables système**, sélectionne **Path** puis clique **Modifier**.
5. Ajoute le chemin où Python est installé, par exemple :

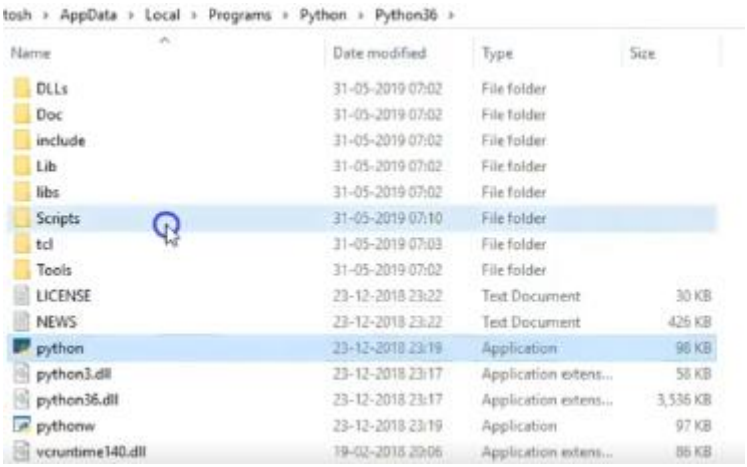
   C:\Users\asus\AppData\Local\Programs\Python\Python313\Scripts

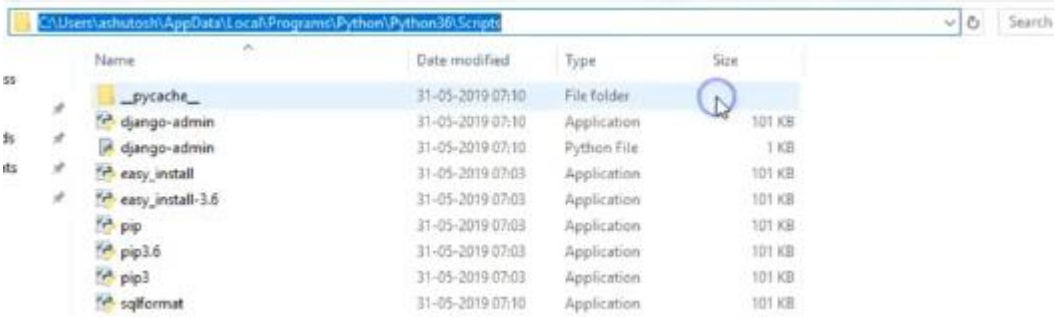





All
Apps
Documents
Settings
Web
Best match
Python 3.6 (64-bit)
App
Run as administrator
Open file location
Pin to Start
Pin to taskbar
Uninstall
Apps
IDLE (Pytho
python-3.6.
python-3.6.6-amd64.exe
Python 3.6 Manuals (64-bit)
Python 3.6 Module Docs (64-bit)
Search the web
python · See web results
Folders (1+)
Store (2)
python


Users › ashutosh › AppData › Roaming › Microsoft › Windows › Start Menu › Programs › Python
Name
Date modified
Type
Size
IDLE (Python 3.6 64-bit)
31-05-2019 07:03
Shortcut
3 KB
Open
Open file location
Run as administrator
Troubleshoot compatibility


tosh › AppData › Local › Programs › Python › Python36 ›
Name
Date modified
Type
Size
DLLs
Doc
include
Lib
libs
Scripts
tcl
Tools
LICENSE
NEWS
python
python3.dll
python36.dll
pythonw
vcruntime140.dll


C:\Users\ashutosh\AppData\Local\Programs\Python\Python36\Scripts
Search
Name
Date modified
Type
Size
__pycache__
31-05-2019 07:10
File folder
django-admin
Application
101 KB
Python File
1 KB
easy_install
easy_install-3.6
pip
pip3.6
pip3
sqlformat

- Un éditeur de texte pour rédiger les script python ou un IDE : Integrated Development Environment tels que :
  - **Sublime Texte** : https://www.sublimetext.com/download
  - **Microsoft Visual Studio Code** : https://code.visualstudio.com
  - **PyCharm** : https://www.jetbrains.com/fr-fr/pycharm/
  - **Anaconda** (Spider ou Jupiter) : https://www.anaconda.com/download

Dans ce TP, je vous propose d'utiliser l'IDE de Microsoft : **Visual Studio Code**.

2. Vérifiez l'installation en ouvrant un terminal :

Pour vérifier Python :

python --version

Pour vérifier pip :

pip --version

2. Configuration de l'environnement de développement

a. Installez VS Code depuis code.visualstudio.com

b. Installez les extensions recommandées :

- Python (Microsoft)
- Django (Baptiste Darthenay)
- SQLite Viewer

3. Créez un dossier pour votre projet

mkdir tp1_django

cd tp1_django

4. **Création d'un environnement virtuel**

Un environnement virtuel permet d'isoler les dépendances et les packages Python localement pour chaque projet.

Commande pour créer un environnement virtuel :

python -m venv .venv

Cela crée un espace isolé afin que chaque projet ait ses propres dépendances

Activer l'environnement virtuel

Selon le système :

- **Linux/Mac** :

  source env_name/bin/activate

- **Windows :**
  syntaxe : env_name\Scripts\activate
  Puisque notre environnement virtuel a été nommé **.venv**

Donc notre commande :

.venv\Scripts\activate

```
C:\Users\asus\tp1_django>python -m venv .venv
```

```
C:\Users\asus\tp1_django>.venv\Scripts\activate

(.venv) C:\Users\asus\tp1_django>
```

4. Installation du framework Django:

- En utilisant toujours un terminal de votre choix, taper les commandes suivantes :

```
(my_environment) … my_project > pip install django
```

- Pour installer une version exacte :

```
(my_environment) … my_project > pip install django==3.1.2
```

pip installera la dernière version disponible de Django, vous pouvez donc voir un numéro de version différent de celui indiqué ci-dessus.

```
(.venv) C:\Users\asus\tp1_django> pip install django
Collecting django
  Downloading django-5.2.7-py3-none-any.whl (8.3 MB)
     |[redacted]| 8.3 MB 94 kB/s
Collecting tzdata
  Downloading tzdata-2025.2-py2.py3-none-any.whl (347 kB)
     |[redacted]| 347 kB 52 kB/s
Collecting sqlparse>=0.3.1
  Using cached sqlparse-0.5.3-py3-none-any.whl (44 kB)
Collecting asgiref>=3.8.1
  Using cached asgiref-3.9.2-py3-none-any.whl (23 kB)
Collecting typing_extensions>=4
  Downloading typing_extensions-4.15.0-py3-none-any.whl (44 kB)
     |[redacted]| 44 kB 64 kB/s
Installing collected packages: typing-extensions, tzdata, sqlparse,
Successfully installed asgiref-3.9.2 django-5.2.7 sqlparse-0.5.3 typ
WARNING: You are using pip version 21.2.4; however, version 25.2 is
You should consider upgrading via the 'C:\Users\asus\tp1_django\.ven
ommand.
```

Pour tester si l'installation a fonctionné, démarrez l'interpréteur interactif Python, puis importer le module django.

Pour connaitre la version de Django installé, exécuter la fonction django.get_version() comme le montre la figure ci-dessous.

```
(.venv) C:\Users\asus\tp1_django>python
Python 3.10.2 (tags/v3.10.2:a58ebcc, Jan 17 2022, 13:59:55) [MSC v.1929 3
2 bit (Intel)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import django
>>> django.get_version()
'5.2.7'
```

N'oubliez pas de quitter l'interpréteur python.

```
>>> exit()
```

Quelques commandes supplémentaires utiles :

1-Vérifier que le module django est bien installé :

Méthode 1 : exécuter la commande :

(venv)C :..>Python -m django --version

Méthode 2 : lancer python, importer django, puis appeler la méthode get-version de django

(venv)C :..>Python
>>>import django
>>>django.get_version()

Création d'un projet Django:

pour créer un projet django ,utiliser la commande suivante :

django-admin startproject tp1 .

```
(.venv) C:\Users\asus\tp1_django>   django-admin startproject tp1 .
```

ou

django-admin startproject tp1 tp1_django

👉 Le **point (.)** à la fin indique à **Django** d'installer le projet **dans le répertoire courant**, et non pas de créer un nouveau sous-dossier supplémentaire.

Démarrer le serveur de développement :

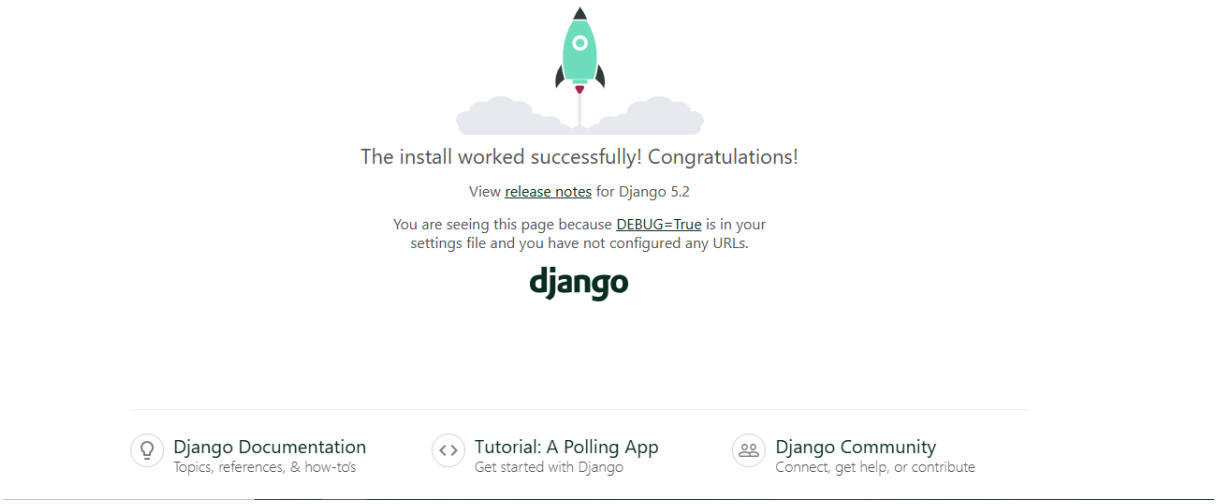
Afin de démarrer le serveur web, on utilise le fichier manage.py en saisissant la commande suivante :

```
(my_environment) … my_project > python manage.py runserver
```

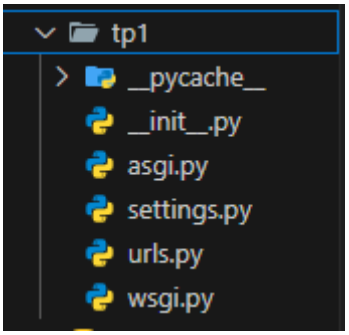
L'étape suivante consiste à tester le bon fonctionnement du serveur web lancé en ouvrant un navigateur web en utilisant l'une des URL suivantes :

- http://localhost:8000
- http://127.0.0.1:8000/

```
(.venv) C:\Users\asus\tp1_django>  python manage.py runserver
```



Structure du projet :



Structure du projet créé :

```
├── manage.py        # Utilitaire en ligne de commande
└── tp1/             # Package Python du projet
    ├── __init__.py  # Indique que c'est un package Python
    ├── settings.py  # Configuration du projet
    ├── urls.py      # Configuration des URLs
    ├── asgi.py      # Point d'entrée ASGI
    └── wsgi.py      # Point d'entrée WSGI
```

Ces fichiers sont :

- **`manage.py`** : un utilitaire en ligne de commande qui vous permet d'interagir avec ce projet Django de différentes façons. Vous trouverez toutes les informations nécessaires sur **`manage.py`** dans django-admin et manage.py.
- **`mysite/`**: un répertoire correspondant au paquet Python effectif de votre projet. C'est le nom du paquet Python que vous devrez utiliser pour importer ce qu'il contient (par ex. **`mysite.urls`**).
- **`mysite/__init__.py`** : un fichier vide qui indique à Python que ce répertoire doit être considéré comme un paquet. Si vous êtes débutant en Python, lisez les informations sur les paquets (en) dans la documentation officielle de Python.
- **`mysite/settings.py`** : réglages et configuration de ce projet Django. Les réglages de Django vous apprendra tout sur le fonctionnement des réglages.
- **`mysite/urls.py`** : les déclarations des URL de ce projet Django, une sorte de « table des matières » de votre site Django. Vous pouvez en lire plus sur les URL dans Distribution des URL.
- **`mysite/asgi.py`** : un point d'entrée pour les serveurs Web compatibles aSGI pour déployer votre projet. Voir Comment déployer avec ASGI pour plus de détails.
- **`mysite/wsgi.py`** : un point d'entrée pour les serveurs Web compatibles WSGI pour déployer votre projet. Voir Comment déployer avec WSGI pour plus de détails.